Unit I – Principles of OOP & Control Structures

Answers: Debugging (Section A) & Pseudo-Code (Section B). All corrected C++ programs were compiled with g++ and run; outputs shown are the real outputs.

Section A – Debugging Questions

Question 1: Operator Precedence Trap & Unsigned Loop

1. Precedence conflict – Line 7 (Logic Defect, compiles fine)

In C++, `==` has **higher** precedence than bitwise `&`. So `num & 1 == 1 && num > 10` is parsed as `(num & (1 == 1)) && (num > 10)` = `(num & 1) && (num > 10)`. It works for mask `1` only by coincidence (`1==1` evaluates to 1). It breaks for any other mask or comparison value, e.g. `num & 2 == 2` becomes `num & 1` (tests the wrong bit), and `num & 4 == 0` becomes `num & 0`, which is always 0. Fix: parenthesise `(num & 1) == 1`.

2. Loop – Line 14 (Logic Defect / infinite loop)

`unsigned char` can never be negative, so `i >= 0` is always true. When `i == 0`, `--i` wraps modulo 256 to 255 (well-defined unsigned wrap-around), and the loop continues forever (255, 254, ..., 0, 255, ...). The `break` is only a patch that hides the flaw. Fix: use a signed counter (`int`).

3. Corrected code

```
#include <iostream>
int main() {
    unsigned char num = 15;
    if (((num & 1) == 1) && (num > 10)) {
        std::cout << "Valid Odd Number: " << static_cast<int>(num) << std::endl;
    } else {
        std::cout << "Condition Failed for: " << static_cast<int>(num) << std::endl;
    }
    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

4. Dry run & output

Step	Evaluation
num = 15 (binary 00001111)	15 & 1 = 1 → (1 == 1) true; 15 > 10 true → "Valid Odd"
Loop i = 3,2,1,0	each prints i; i becomes −1 → `i >= 0` false → exit

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution & Shadowing

1. Syntax error – Line 11 (Compilation Error)

`:taxRate` uses a single colon. The scope-resolution operator is the two-character token `::`. A lone `:` is not valid in an expression, so the compiler reports a parse error (expected primary-expression before ':'). Fix: `::taxRate`.

2. What `::taxRate` means on Line 16 (Semantic/Logic Defect)

The unary `::` always looks up the name in the **global (file) namespace**. So on Line 16 it resolves to the global `double taxRate = 0.18`, **not** the local 0.05 as the developer wanted. No, `::` **cannot** reach an enclosing function/block scope. C++ has no operator for accessing a shadowed variable in an enclosing local scope. Once the inner `int taxRate = 12` is declared, the outer local `taxRate` (0.05) is hidden for the rest of that block. Only global names can be recovered with `::`.

Solution: give distinct names (or copy the outer local into a differently-named variable before entering the block).

3. Corrected code

```
#include <iostream>
double globalTaxRate = 0.18;          // global standard rate
int main() {
    double itemPrice = 1000.0;
    double localTaxRate = 0.05;       // subsidised rate
    std::cout << "Local Rate: " << localTaxRate << std::endl;
    double totalWithGlobalTax = itemPrice + (itemPrice * ::globalTaxRate);
    {
        int corporateTaxRate = 12;
        double totalWithLocalTax = itemPrice + (itemPrice * localTaxRate);
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
        std::cout << "Corporate Rate: " << corporateTaxRate << "%" << std::endl;
    }
    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

4. Output

```
Local Rate: 0.05
Inner total: 1050
Corporate Rate: 12%
Final Global Calculation: 1180
```

Check: 1000 + 1000×0.05 = 1050; 1000 + 1000×0.18 = 1180.

Question 3: Switch-Case Initialization & Illegal Case Types

1. Cross-initialization (Compilation Error – `case 2`; same issue at `case 1`)

All `case` labels share the single scope of the switch body. A variable declared with an initializer in `case 1` (`offset`) or `case 2` (`multiplier`) is still in scope at later labels, but jumping directly to a later label would bypass its initialization. C++ forbids jumping past a declaration with initialization ("jump to case label crosses initialization of 'int multiplier'"). Fix: enclose each case body in its own braces `{ }`.

2. `case 4.5:` (Compilation Error)

A case label must be an **integral constant expression**, and the switch condition must be of integral or enumeration type. Legal: `char, signed/unsigned char, short, int, long, long long, bool`, enums (and class types convertible to these). Illegal: `float, double`, strings, pointers. 4.5 is a floating literal, so it is rejected (use an integer like `case 4:`).

3. Missing `break` (Logic Defect)

With `tokenType == 2`, `tokenValue = 20 × 5 = 100`, then control **falls through** into `case 3` and executes `tokenValue += 100`, giving 200 instead of 100.

4. Corrected code

```
#include <iostream>
int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;
    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;                  // prevents fall-through
        }
        case 3:
            tokenValue += 100.0f;
            break;
        case 4:                     // integral label
            std::cout << "Float token detected" << std::endl;
            break;
        default:
            std::cout << "Unknown token" << std::endl;
    }
    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Dry run: tokenType = 2 → matches case 2 → multiplier = 5 → tokenValue = 100 → break.

```
Calculated Value: 100
```

Question 4: Reference Initialization & Constant Binding

1. Violations (all Compilation Errors)

Line	Statement	Rule violated
7	`int &refA;`	A reference must be initialized at its declaration; it cannot be left unbound or re-seated later. (`refA = a;` would assign through it anyway, not bind.)
11	`int &refLiteral = 100;`	A non-const lvalue reference cannot bind to an rvalue (literal has no address/identity).
14	`int &refInt = pi;`	`double → int` conversion creates a temporary rvalue; a non-const reference cannot bind to it (and would not alias `pi` anyway).
18	`constRef = constRef + 10;`	A reference to const gives read-only access; assigning through it is an illegal modification of a const object.

2. Lifetime extension

When a `const T&` is bound to an rvalue or to a value needing conversion, the compiler creates an unnamed temporary and binds the reference to it. The C++ standard **extends the temporary's lifetime** to match the reference's own scope (instead of destroying it at the end of the full expression). So `const int &r = 50;` behaves like `const int tmp = 50; const int &r = tmp;`, and `const int &r = pi;` binds to a temporary int holding 3 (truncated). Const is required because the compiler knows the temp is read-only, so a silent modification of a throw-away value cannot happen.

3. Corrected code

```
#include <iostream>
int main() {
    int a = 25;
    int &refA = a;                    // bound at declaration
    const int &refLiteral = 100;      // const ref -> temporary, lifetime extended
    double pi = 3.14159;
    const int &refInt = pi;           // temporary int(3), lifetime extended
    const int &constRef = 50;
    int updated = constRef + 10;      // read only; store result in a new variable
    std::cout << a << " " << refA << std::endl;
    std::cout << refLiteral << " " << refInt << " " << constRef << " " << updated << std::endl;
    return 0;
}
```

4. Output

```
25 25
100 3 50 60
```

The first line is the original program's output; the second line shows the extra values (3.14159 truncates to 3).

Question 5: Dynamic Array & Stray Semicolon

1. Semicolon on Line 10 (Compilation Error as written)

`for (...);` makes the loop body an **empty statement**. The loop just spins and increments `i` to 5, doing nothing. The following `{ readings[i] = ...; }` is then an ordinary, independent block. Because `int i` was declared in the for-header, its scope ends at the semicolon, so `i` is **undeclared** inside the block → "'i' was not declared in this scope". (If `i` were declared outside, the

block would run once with `i == 5`, writing `readings[5]`, a **heap buffer overflow (undefined behaviour)**, while `readings[0..4]` stay uninitialized, so the sum would be garbage.)

2. Line 21 (Undefined Behaviour)

`new int[size]` must be released with `delete[]`. `delete readings;` uses the scalar form on an array: undefined behaviour (typically heap corruption; for types with destructors, only one destructor is called). `delete` frees a single object created with `new`; `delete[]` destroys all elements of an array created with `new[]` and releases the whole block.

3. Corrected code

```
#include <iostream>
int main() {
    int size = 5;
    int *readings = new int[size];
    for (int i = 0; i < size; ++i) {      // semicolon removed
        readings[i] = (i + 1) * 10;
    }
    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }
    std::cout << "Total: " << total << std::endl;
    delete[] readings;                      // matches new[]
    return 0;
}
```

4. Trace

i	readings[i]	running total
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

```
Total: 150
```

Section B – Pseudo-Code Questions

Question 1: WaterConsumer – Procedural to OOP

Design overview

- **Encapsulation:** all five attributes are PRIVATE; only methods change them.
- **Data hiding / validation:** the constructor-like `Initialize` rejects negative readings or current < previous and sets an error flag (`isValid`), so an invalid object never computes a bill.
- **Abstraction:** the caller just calls CalculateConsumption → ComputeBill → DisplayBill; tier logic is hidden.
- **Assumption:** the 10% surcharge applies to the entire bill only when units exceed 300 (it is stated under that tier).

Structured pseudo-code

```
CLASS WaterConsumer
  PRIVATE:
    STRING  consumerID
    REAL    previousReading, currentReading
    REAL    unitsConsumed, totalBill
    BOOLEAN isValid

  PUBLIC:
    CONSTRUCTOR WaterConsumer()
      SET consumerID = ""
      SET previousReading = 0, currentReading = 0
      SET unitsConsumed = 0, totalBill = 0
      SET isValid = FALSE
    ENDCONSTRUCTOR

    METHOD Initialize(STRING id, REAL prev, REAL curr)
      IF prev < 0 OR curr < 0 THEN
        SET isValid = FALSE
        DISPLAY "Error: readings must be non-negative"
      ELSE
        IF curr < prev THEN
          SET isValid = FALSE
          DISPLAY "Error: current reading is less than previous reading"
        ELSE
          SET consumerID = id
          SET previousReading = prev
          SET currentReading = curr
          SET isValid = TRUE
        ENDIF
      ENDIF
    ENDMETHOD

    METHOD CalculateConsumption()
      IF isValid = TRUE THEN
        SET unitsConsumed = currentReading - previousReading
      ELSE
        SET unitsConsumed = 0
      ENDIF
    ENDMETHOD

    METHOD ComputeBill()
      REAL bill
      IF isValid = FALSE THEN
        SET totalBill = 0
      ELSE
        IF unitsConsumed <= 100 THEN
          SET bill = unitsConsumed * 1.50
        ELSE
          IF unitsConsumed <= 300 THEN
            SET bill = (100 * 1.50) + ((unitsConsumed - 100) * 2.50)
          ELSE
            SET bill = (100 * 1.50) + (200 * 2.50) + ((unitsConsumed - 300) * 4.00)
            SET bill = bill + (bill * 0.10)
          ENDIF
        ENDIF
        SET totalBill = bill
      ENDIF
    ENDMETHOD

    METHOD DisplayBill()
      IF isValid = FALSE THEN
        DISPLAY "Invalid consumer record - no bill generated"
      ELSE
        DISPLAY "Consumer ID : ", consumerID
```

```
        DISPLAY "Previous    : ", previousReading
        DISPLAY "Current     : ", currentReading
        DISPLAY "Units used  : ", unitsConsumed
        DISPLAY "Total Bill  : $", totalBill
      ENDIF
    ENDMETHOD
ENDCLASS

BEGIN Main
  WaterConsumer c
  c.Initialize("WC-1001", 1450.0, 1820.0)
  c.CalculateConsumption()
  c.ComputeBill()
  c.DisplayBill()
END
```

Trace table (prev = 1450.0, curr = 1820.0)

Step	Computation	Value
Validate	1450 ≥ 0, 1820 ≥ 0, 1820 ≥ 1450	isValid = TRUE
Consumption	1820 − 1450	370.0 units
Branch	370 > 300	third tier
Tier 1	100 × 1.50	150.00
Tier 2	200 × 2.50	500.00
Tier 3	(370 − 300) = 70 × 4.00	280.00
Subtotal	150 + 500 + 280	930.00
Surcharge	10% × 930	93.00
Total bill	930 + 93	**$1,023.00**

Question 2: Multi-Tier Tax & Precision Cast Engine

Design overview

- Clamp each deduction to its statutory cap with IF (MIN logic), then clamp taxable income at 0.
- Progressive tax via a nested IF-ELSE ladder (each slab taxes only the portion in that slab, so lower slabs are added in full).
- Cess = 4% of raw tax; final REAL → INTEGER conversion by adding 0.5 then truncating (round to nearest).

Structured pseudo-code

```
BEGIN TaxCalculationEngine
  REAL    grossAnnualSalary, deduction80C, deductionHealthInsurance
  REAL    eligible80C, eligibleHealth, totalDeductions
  REAL    taxableIncome, rawTax, cess, finalTaxReal
  INTEGER finalTaxRounded
  CONSTANT REAL CAP_80C = 150000
  CONSTANT REAL CAP_HEALTH = 25000

  READ grossAnnualSalary, deduction80C, deductionHealthInsurance

  IF deduction80C > CAP_80C THEN
    SET eligible80C = CAP_80C
  ELSE
    SET eligible80C = deduction80C
  ENDIF

  IF deductionHealthInsurance > CAP_HEALTH THEN
    SET eligibleHealth = CAP_HEALTH
  ELSE
    SET eligibleHealth = deductionHealthInsurance
  ENDIF

  SET totalDeductions = eligible80C + eligibleHealth
  SET taxableIncome = grossAnnualSalary - totalDeductions
  IF taxableIncome < 0 THEN
    SET taxableIncome = 0
  ENDIF

  IF taxableIncome <= 250000 THEN
    SET rawTax = 0
  ELSE
    IF taxableIncome <= 500000 THEN
      SET rawTax = (taxableIncome - 250000) * 0.05
    ELSE
      IF taxableIncome <= 1000000 THEN
        SET rawTax = (250000 * 0.05) + (taxableIncome - 500000) * 0.20
      ELSE
        SET rawTax = (250000 * 0.05) + (500000 * 0.20) + (taxableIncome - 1000000) * 0.30
      ENDIF
    ENDIF
  ENDIF

  SET cess = rawTax * 0.04
  SET finalTaxReal = rawTax + cess
  SET finalTaxRounded = INTEGER(finalTaxReal + 0.5)   // explicit cast

  DISPLAY "Taxable income : ", taxableIncome
  DISPLAY "Total tax      : ", finalTaxRounded
END
```

Trace table (gross = 1,200,000; 80C = 180,000; Health = 30,000)

Step	Computation	Value
eligible80C	180,000 > 150,000 → cap	150,000
eligibleHealth	30,000 > 25,000 → cap	25,000
totalDeductions	150,000 + 25,000	175,000
taxableIncome	1,200,000 − 175,000	1,025,000
Slab selection	1,025,000 > 1,000,000	top slab
Slab 250k–500k	250,000 × 5%	12,500
Slab 500k–1M	500,000 × 20%	100,000
Above 1M	25,000 × 30%	7,500
rawTax	12,500 + 100,000 + 7,500	120,000
cess	4% × 120,000	4,800
finalTaxReal	120,000 + 4,800	124,800.00
Cast	INTEGER(124,800.00 + 0.5)	**124,800**

Question 3: Bitwise Mask Validation – UAV Status Register

1. Hexadecimal constants

```
CONSTANT GYRO_LOCKED  = 0x01      CONSTANT LOW_BATTERY  = 0x10
CONSTANT ACCEL_CALIB  = 0x02      CONSTANT OVER_TEMP    = 0x20
CONSTANT BARO_ONLINE  = 0x04      CONSTANT RADIO_LINK   = 0x40   (non-blocking)
CONSTANT GPS_FIX      = 0x08      CONSTANT FATAL_FAIL   = 0x80

CONSTANT MANDATORY_MASK = GYRO_LOCKED | ACCEL_CALIB | BARO_ONLINE | GPS_FIX   = 0x0F
CONSTANT INHIBIT_MASK   = LOW_BATTERY | OVER_TEMP | FATAL_FAIL                = 0xB0
```

Design overview

- `status AND INHIBIT_MASK` isolates the inhibitor bits; non-zero means a fault, and each fault is identified with a single-bit AND.
- `status AND MANDATORY_MASK` isolates sensors; `XOR MANDATORY_MASK` yields exactly the missing sensors (bits that should be 1 but are 0).
- No loops over bit positions. Bit 6 is ignored because neither mask contains it.

2. Structured pseudo-code

```
BEGIN CheckFlightReadiness(INTEGER statusRegister)   // 8-bit unsigned
  INTEGER faultBits, sensorBits, missingMask
  BOOLEAN ready
  SET ready = TRUE

  SET faultBits = statusRegister AND INHIBIT_MASK
  IF faultBits NOT= 0 THEN
    SET ready = FALSE
    IF (faultBits AND LOW_BATTERY) NOT= 0 THEN
      DISPLAY "FAULT: Low battery warning (bit 4)"
    ENDIF
    IF (faultBits AND OVER_TEMP) NOT= 0 THEN
      DISPLAY "FAULT: Over-temperature warning (bit 5)"
    ENDIF
    IF (faultBits AND FATAL_FAIL) NOT= 0 THEN
      DISPLAY "FAULT: Fatal actuator failure (bit 7)"
    ENDIF
  ENDIF

  SET sensorBits  = statusRegister AND MANDATORY_MASK
  SET missingMask = sensorBits XOR MANDATORY_MASK
  IF missingMask NOT= 0 THEN
    SET ready = FALSE
    DISPLAY "MISSING SENSORS, mask = ", missingMask
    IF (missingMask AND GYRO_LOCKED) NOT= 0 THEN DISPLAY "  Gyroscope not locked (bit 0)" ENDIF
    IF (missingMask AND ACCEL_CALIB) NOT= 0 THEN DISPLAY "  Accelerometer not calibrated (bit
1)" ENDIF
    IF (missingMask AND BARO_ONLINE) NOT= 0 THEN DISPLAY "  Barometer offline (bit 2)" ENDIF
    IF (missingMask AND GPS_FIX)     NOT= 0 THEN DISPLAY "  No GPS fix (bit 3)" ENDIF
  ENDIF

  IF ready = TRUE THEN
    DISPLAY "FLIGHT READY"
    IF (statusRegister AND RADIO_LINK) = 0 THEN
      DISPLAY "Note: radio link inactive (non-blocking)"
    ENDIF
  ELSE
    DISPLAY "NOT READY FOR FLIGHT"
  ENDIF
END
```

3. Trace

Step	0x4F = 0100 1111	0x3B = 0011 1011
status AND 0xB0	0x4F & 0xB0 = 0x00	0x3B & 0xB0 = 0x30 (bits 4 and 5)
Inhibitor result	none → ready stays TRUE	FAULT: Low battery (bit 4) and FAULT: Over-temperature (bit 5) → ready = FALSE
status AND 0x0F	0x0F	0x0B
XOR 0x0F (missing)	0x00	0x04 → Barometer offline (bit 2)
Radio (0x40)	set (active)	clear (not tested, non-blocking)
Result	**FLIGHT READY**	**NOT READY FOR FLIGHT**

> Note: 0x3B actually has *both* bit 4 (low battery) and bit 5 (over-temperature) set (0x3B = 0x20 + 0x10 + 0x08 + 0x02 + 0x01), so the algorithm correctly reports two faults, not just low battery.

Question 4: Divisor Engine & Perfect/Abundant/Deficient Classification

Design overview

- Input validation uses a backward `GOTO ReadInput` jump while N ≤ 0.
- N = 1 handled explicitly (no proper divisors, sum = 0 → Deficient).
- Only candidates 2 … ⌊√N⌋ are tested (O(√N)); each hit adds both *d* and N/d; when d = N/d only one is added. The sum starts at 1 (1 is always a proper divisor for N > 1).
- `CONTINUE` skips non-divisors.

Structured pseudo-code

```
BEGIN DivisorAndPrimeEngine
  INTEGER N, d, limit, pairD, sumOfDivisors

  ReadInput:
  DISPLAY "Enter a positive integer N: "
  READ N
  IF N <= 0 THEN
    DISPLAY "Error: N must be positive"
    GOTO ReadInput
  ENDIF

  IF N = 1 THEN
    DISPLAY "1 has no proper divisors (sum = 0): Deficient"
    GOTO Finish
  ENDIF

  SET sumOfDivisors = 1
  SET limit = FLOOR(SQRT(N))
  FOR d = 2 TO limit STEP 1 DO
    IF N MOD d NOT= 0 THEN
      CONTINUE
    ENDIF
    SET pairD = N / d                 // integer division
    IF pairD = d THEN
      SET sumOfDivisors = sumOfDivisors + d
    ELSE
      SET sumOfDivisors = sumOfDivisors + d + pairD
    ENDIF
  ENDFOR

  IF sumOfDivisors = N THEN
    DISPLAY N, " is a Perfect number"
  ELSE
    IF sumOfDivisors > N THEN
      DISPLAY N, " is an Abundant number"
    ELSE
      DISPLAY N, " is a Deficient number"
    ENDIF
  ENDIF

  Finish:
END
```

Trace table

N	limit	d	N MOD d	N / d	Action	sumOfDivisors
28	5	(init)	–	–	start	1
		2	0	14	add 2 + 14	17
		3	1	–	CONTINUE	17
		4	0	7	add 4 + 7	28
		5	3	–	CONTINUE	28
28 = 28 → **Perfect** (divisors 1, 2, 4, 7, 14)						
12	3	(init)	–	–	start	1
		2	0	6	add 2 + 6	9
		3	0	4	add 3 + 4	16
16 > 12 → **Abundant** (divisors 1, 2, 3, 4, 6)						

Question 5: ATM Controller with Sentinel Loop

Design overview

- Authentication: counted WHILE loop with up to 3 attempts; the card locks and the program ends if all fail.
- Session: sentinel-controlled WHILE loop (sentinel = menu option 4); menu dispatch with a CASE (switch) structure and a default branch.
- Withdrawal validation: amount > 0, amount ≤ balance, `amount MOD 100 = 0`.
- **Assumption:** opening balance = $1,000 (not given in the question); the PIN is 4821.

Structured pseudo-code

```
BEGIN ATM_SessionController
  CONSTANT INTEGER CORRECT_PIN = 4821
  CONSTANT INTEGER MAX_ATTEMPTS = 3
  INTEGER pinEntered, attempts, choice
  REAL    balance, amount
  BOOLEAN authenticated, running

  SET balance = 1000.00
  SET attempts = 0
  SET authenticated = FALSE

  WHILE attempts < MAX_ATTEMPTS AND authenticated = FALSE DO
    DISPLAY "Enter 4-digit PIN: "
    READ pinEntered
    SET attempts = attempts + 1
    IF pinEntered = CORRECT_PIN THEN
      SET authenticated = TRUE
    ELSE
      DISPLAY "Incorrect PIN. Attempts left: ", MAX_ATTEMPTS - attempts
    ENDIF
  ENDWHILE

  IF authenticated = FALSE THEN
    DISPLAY "Card locked. Please contact your bank."
    END                                  // terminate immediately
  ENDIF

  SET running = TRUE
  WHILE running = TRUE DO
    DISPLAY "1: Balance  2: Deposit  3: Withdraw  4: Exit"
    READ choice
    CASE choice OF
      1:  DISPLAY "Balance: $", balance
      2:  DISPLAY "Enter deposit amount: "
          READ amount
          IF amount > 0 THEN
            SET balance = balance + amount
            DISPLAY "Deposited. New balance: $", balance
          ELSE
            DISPLAY "Invalid deposit amount"
          ENDIF
      3:  DISPLAY "Enter withdrawal amount: "
          READ amount
          IF amount <= 0 THEN
            DISPLAY "Invalid amount"
          ELSE
            IF amount > balance THEN
              DISPLAY "Insufficient funds"
            ELSE
              IF amount MOD 100 NOT= 0 THEN
                DISPLAY "Amount must be a multiple of 100"
              ELSE
                SET balance = balance - amount
                DISPLAY "Please collect cash. Balance: $", balance
              ENDIF
            ENDIF
          ENDIF
      4:  DISPLAY "Thank you. Goodbye."
          SET running = FALSE
      DEFAULT: DISPLAY "Invalid option. Choose 1-4."
    ENDCASE
  ENDWHILE
END
```

Dry-run trace (opening balance $1,000)

#	Input	Condition checks	attempts / balance	Output
1	PIN = 1234	1234 ≠ 4821	attempts = 1	Incorrect PIN. Attempts left: 2
2	PIN = 4821	4821 = 4821 → authenticated	attempts = 2	(enter session loop)
3	Menu 3, amount 250	250 > 0; 250 ≤ 1000; 250 MOD 100 = 50 ≠ 0	balance 1000	Amount must be a multiple of 100 (declined)
4	Menu 3, amount 300	300 > 0; 300 ≤ 1000; 300 MOD 100 = 0	balance 1000 − 300 = 700	Please collect cash. Balance: $700
5	Menu 4	sentinel → running = FALSE	balance 700	Thank you. Goodbye.